



Red Hat OpenShift AI Self-Managed 3.5

Configuring your model-serving platform

Configure your model-serving platform in Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.5 Configuring your model-serving platform

Configure your model-serving platform in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

As a Red Hat OpenShift AI administrator, you can configure your model serving platform in Red Hat OpenShift AI Self-Managed.

Table of Contents

PREFACE	3
CHAPTER 1. MODEL-SERVING PLATFORMS	4
1.1. MODEL SERVING FUNDAMENTALS	4
1.1.1. Model serving platform	4
1.1.2. NVIDIA NIM model serving platform	4
1.2. MODEL-SERVING RUNTIMES	5
1.2.1. ServingRuntime	5
1.2.2. InferenceService	6
1.3. VLLM RUNTIME SUPPORT LEVELS	7
1.4. LIMITED-SUPPORT RUNTIME VISIBILITY AND GATING	9
1.5. MODEL-SERVING RUNTIMES FOR ACCELERATORS	10
1.5.1. NVIDIA GPUs	10
1.5.2. Intel Gaudi accelerators	10
1.5.3. AMD GPUs	11
1.5.4. IBM Spyre AI accelerators on x86_64, IBM Power, and IBM Z	11
1.5.5. Supported model-serving runtimes	11
1.5.6. Tested and verified model-serving runtimes	12
CHAPTER 2. CONFIGURE MODEL SERVERS	13
2.1. ENABLE THE MODEL SERVING PLATFORM	13
2.2. ENABLE SPECULATIVE DECODING AND MULTI-MODAL INFERENCING	13
2.3. ADDING A CUSTOM MODEL-SERVING RUNTIME	15
2.4. ADD A TESTED AND VERIFIED RUNTIME	17
2.5. ENABLE LIMITED-SUPPORT SERVING RUNTIMES FROM THE DASHBOARD	23
2.6. ENABLE LIMITED-SUPPORT SERVING RUNTIMES USING GITOPS	25
2.7. LIMITED-SUPPORT RUNTIME STATE AND BACKWARD COMPATIBILITY	27
2.8. ENABLE THE NVIDIA NIM MODEL SERVING PLATFORM	30
CHAPTER 3. CUSTOMIZE MODEL DEPLOYMENTS	32
3.1. CUSTOMIZE THE PARAMETERS OF A DEPLOYED MODEL-SERVING RUNTIME	32
3.2. CUSTOMIZABLE MODEL SERVING RUNTIME PARAMETERS	33
3.3. CUSTOMIZE THE VLLM MODEL-SERVING RUNTIME	34
3.4. SET A DEFAULT CLUSTER-WIDE DEPLOYMENT STRATEGY	36
CHAPTER 4. CACHE MODELS ON LOCAL STORAGE	37
4.1. MODEL CACHE FOR FASTER INFERENCE STARTUP	37
4.1.1. Cold start latency and model caching	37
4.1.2. Custom resources	37
4.1.3. Cluster-scoped and namespace-scoped caching	38
4.2. ENABLE MODEL CACHING	38
4.3. CONFIGURE NODE GROUPS FOR MODEL CACHING	40
4.4. CONFIGURE MODEL CACHE DOWNLOAD CREDENTIALS	42
4.5. MODEL CACHE CONFIGURATION PARAMETERS	43
4.5.1. DataScienceCluster model cache parameters	44
4.5.2. Model size reference for capacity planning	45
4.6. TROUBLESHOOT MODEL CACHING ERRORS	45

PREFACE

Configure the model-serving platform in OpenShift AI to deploy and serve trained models.

CHAPTER 1. MODEL-SERVING PLATFORMS

As an OpenShift AI administrator, you can enable your preferred serving platform and make it available for serving models. You can also add a custom or a tested and verified model-serving runtime.

1.1. MODEL SERVING FUNDAMENTALS

When you serve a model, you upload a trained model into Red Hat OpenShift AI for querying, which allows you to integrate your trained models into intelligent applications.

You can upload a model to an S3-compatible object storage, persistent volume claim, or Open Container Initiative (OCI) image. You can then access and train the model from your project workbench. After training the model, you can serve or deploy the model using a model-serving platform.

Serving or deploying the model makes the model available as a service, or model runtime server, that you can access using an API. You can then access the inference endpoints for the deployed model from the dashboard and see predictions based on data inputs that you provide through API calls. Querying the model through the API is also called model inferencing.

You can also serve models on the NVIDIA NIM model serving platform. The model-serving platform that you choose depends on your business needs:

- If you want to deploy each model on its own runtime server, select the **Model serving platform**. The model serving platform is recommended for production use.
- If you want to use NVIDIA Inference Microservices (NIMs) to deploy a model, select the **NVIDIA NIM-model serving platform**.

1.1.1. Model serving platform

You can deploy each model from a dedicated model server, which can help you deploy, monitor, scale, and maintain models that require increased resources. Based on the KServe component, this model serving platform is ideal for serving large models.

The model serving platform is helpful for use cases such as:

- Large language models (LLMs)
- Generative AI

1.1.2. NVIDIA NIM model serving platform

You can deploy models using NVIDIA Inference Microservices (NIM) on the NVIDIA NIM model serving platform.

NVIDIA NIM, part of NVIDIA AI Enterprise, is a set of microservices designed for secure, reliable deployment of high performance AI model inferencing across clouds, data centers and workstations.

NVIDIA NIM inference services are helpful for use cases such as:

- Using GPU-accelerated containers inferencing models optimized by NVIDIA
- Deploying generative AI for virtual screening, content generation, and avatar creation

The NVIDIA NIM model serving platform is based on the model serving platform. To use the NVIDIA NIM model serving platform, you must first install the model serving platform.

Additional resources

- [KServe](#)
- [Installing and managing Red Hat OpenShift AI components](#)

1.2. MODEL-SERVING RUNTIMES

You can serve models on the single-model serving platform by using model-serving runtimes. The configuration of a model-serving runtime is defined by the **ServingRuntime** and **InferenceService** custom resource definitions (CRDs).

1.2.1. ServingRuntime

The **ServingRuntime** CRD creates a serving runtime, an environment for deploying and managing a model. It creates the templates for pods that dynamically load and unload models of various formats and also exposes a service endpoint for inferencing requests.

The following YAML configuration is an example of the **vLLM ServingRuntime for KServe** model-serving runtime. The configuration includes various flags, environment variables and command-line arguments.

```
apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  annotations:
    opendatahub.io/recommended-accelerators: '["nvidia.com/gpu"]'
    openshift.io/display-name: vLLM ServingRuntime for KServe
  labels:
    opendatahub.io/dashboard: "true"
  name: vllm-runtime
spec:
  annotations:
    prometheus.io/path: /metrics
    prometheus.io/port: "8080"
  containers :
    - args:
      - --port=8080
      - --model=/mnt/models
      - --served-model-name={{.Name}}
      command:
      - python
      - '-m'
      - vllm.entrypoints.openai.api_server
      env:
      - name: HF_HOME
        value: /tmp/hf_home
      image:
quay.io/modh/vllm@sha256:8a3dd8ad6e15fe7b8e5e471037519719d4d8ad3db9d69389f2beded36a6f5
b21
      name: kserve-container
      ports:
```

```

      - containerPort: 8080
        protocol: TCP
    multiModel: false
    supportedModelFormats:
      - autoSelect: true
        name: vLLM

```

Where:

- **opendatahub.io/recommended-accelerators:** The recommended accelerator to use with the runtime.
- **openshift.io/display-name:** The name with which the serving runtime is displayed.
- **prometheus.io/path:** The endpoint used by Prometheus to scrape metrics for monitoring.
- **prometheus.io/port:** The port used by Prometheus to scrape metrics for monitoring.
- **--model:** The path to where the model files are stored in the runtime container.
- **--served-model-name:** Passes the model name that is specified by the **{{.Name}}** template variable inside the runtime container specification to the runtime environment. The **{{.Name}}** variable maps to the **spec.predictor.name** field in the **InferenceService** metadata object.
- **command:** The entrypoint command that starts the runtime container.
- **image:** The runtime container image used by the serving runtime. This image differs depending on the type of accelerator used.
- **multiModel:** Specifies that the runtime is used for single-model serving.
- **supportedModelFormats:** Specifies the model formats supported by the runtime.

1.2.2. InferenceService

The **InferenceService** CRD creates a server or inference service that processes inference queries, passes it to the model, and then returns the inference output.

The inference service also performs the following actions:

- Specifies the location and format of the model.
- Specifies the serving runtime used to serve the model.
- Enables the passthrough route for gRPC or REST inference.
- Defines HTTP or gRPC endpoints for the deployed model.

The following example shows the InferenceService YAML configuration file that is generated when deploying a granite model with the vLLM runtime:

```

apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  annotations:
    openshift.io/display-name: granite

```

```

serving.knative.openshift.io/enablePassthrough: 'true'
sidecar.istio.io/inject: 'true'
sidecar.istio.io/rewriteAppHTTPProbers: 'true'
name: granite
labels:
  opendatahub.io/dashboard: 'true'
spec:
  predictor:
    maxReplicas: 1
    minReplicas: 1
    model:
      modelFormat:
        name: vLLM
      name: ""
    resources:
      limits:
        cpu: '6'
        memory: 24Gi
        nvidia.com/gpu: '1'
      requests:
        cpu: '1'
        memory: 8Gi
        nvidia.com/gpu: '1'
    runtime: vLLM ServingRuntime for KServe
  storage:
    key: aws-connection-my-storage
    path: models/granite-7b-instruct/
  tolerations:
    - effect: NoSchedule
      key: nvidia.com/gpu
      operator: Exists

```

Additional resources

- [Serving Runtimes](#)

1.3. VLLM RUNTIME SUPPORT LEVELS

vLLM runtimes and accelerator configurations in OpenShift AI have three support levels: supported (GA), limited support (fast builds), and unsupported (custom), enabling you to balance production stability with access to newer features.

OpenShift AI provides vLLM runtimes at three distinct support levels, each suited to different use cases and risk tolerance levels.

Limited-support runtimes, also called fast builds, are Red Hat-built vLLM container images published to **registry.redhat.io/rhaii-early-access**. These runtimes are released more frequently than standard GA vLLM runtimes, providing faster access to upstream vLLM features and model format support.

Fast builds differ from custom runtimes in an important way:

- **Limited-support runtimes** are built and maintained by Red Hat with a defined support coverage window (1 month after release).
- **Custom runtimes** are created and maintained by users with no Red Hat support.

Support levels

- **Supported (GA):**

Standard vLLM runtimes with full Red Hat support coverage for the entire OpenShift AI product lifecycle. These runtimes are stable, tested, and recommended for production workloads.

Identification: "Pre-installed" label with version badge (for example, "0.6.1"). No "Limited support" or "fast-N" badges.

- **Limited support (fast builds):**

Red Hat-built vLLM container images with support coverage for 1 month after release, where available in your version of OpenShift AI. These runtimes provide faster access to upstream vLLM features and model formats.

Identification: "Pre-installed" label with "Limited support" badge, version badge (for example, "0.6.1"), and "fast-N" badge (for example, "fast-1").

- **Unsupported (custom):**

User-created serving runtimes with no Red Hat support coverage. Users are responsible for building, maintaining, and licensing these runtimes.

Identification: No "Pre-installed" label.

Badge indicators

The OpenShift AI Dashboard displays badges to help you identify runtime and accelerator configuration support levels. These badges appear in two locations:

- In the administrator view, these badges appear in **Settings → Model resources and operations → Serving runtimes** (for ServingRuntime templates) or **Settings → Model resources and operations → LLM accelerator configurations** (for LLMInferenceServiceConfigs, requires **vLLMDeploymentOnMaaS** feature flag).
- In the data scientist view, these badges appear in the model deployment wizard **Serving runtime template** dropdown (for ServingRuntimes) or **Accelerator configuration** dropdown (for LLMInferenceServiceConfigs).

Badge	Color	Meaning
Limited support	Orange	1-month support window; administrator has accepted risk
Version badge (for example, "0.6.1")	Blue	Upstream vLLM version
fast-N (for example, "fast-1")	Yellow	Fast build iteration number
Pre-installed	Blue	Red Hat-provided runtime (GA or limited-support)

Fast build versioning

Fast builds use a versioning scheme indicated by the annotation **opendatahub.io/fast-version**. Current fast builds use sequential numeric versions:

- **fast-1** - First fast build iteration
- **fast-2** - Second fast build iteration
- **fast-N** - Nth fast build iteration

Each fast build corresponds to a specific upstream vLLM version, which is displayed in the OpenShift AI Dashboard with a version badge (for example, "0.6.1").

Serving architecture support

Limited-support annotations work identically for both OpenShift AI serving architectures:

- **ServingRuntime Templates** - Traditional **KServe** v1alpha1 model serving
- **LLMInferenceServiceConfig** - **KServe GenAI** architecture for LLM-specific serving
Administrators can accept limited-support runtimes via the Dashboard or by adding annotations to GitOps manifests, regardless of which serving architecture they use.

Additional resources

- [Understanding unsupported runtime visibility and gating](#)
- [vLLM runtime badge reference](#)
- [Adding a custom model-serving runtime](#)
- [What is vLLM?](#)

1.4. LIMITED-SUPPORT RUNTIME VISIBILITY AND GATING

Limited-support vLLM runtimes are hidden from data scientists until an administrator explicitly accepts the risk through an annotation-based gating mechanism. This prevents accidental deployment on limited-support runtimes without organizational approval.

OpenShift AI uses two annotations to control runtime visibility and acceptance:

- **opendatahub.io/support-status: unsupported** - Marks a runtime as limited-support, requiring acceptance
- **opendatahub.io/unsupported-status-accepted: "true"** - Indicates administrator acceptance

Acceptance workflows

Administrators can accept limited-support runtimes through two workflows:

- **Dashboard workflow:**
Go to **Settings** → **Model resources and operations** → **Serving runtimes** (or **LLM accelerator configurations** for LLMInferenceServiceConfigs) and toggle the runtime or configuration to enabled. A risk acceptance modal is displayed with the message: "The support coverage for this runtime is limited to 1 month after release." When you select "I understand" and click **Enable**, you automatically add the **opendatahub.io/unsupported-status-accepted: "true"** annotation.
- **GitOps workflow:**

Add both **opendatahub.io/support-status: unsupported** and **opendatahub.io/unsupported-status-accepted: "true"** annotations to the **metadata.annotations** section of a **ServingRuntime** or **LLMInferenceServiceConfig** YAML manifest. Commit and push to your GitOps repository. The runtime becomes visible immediately after sync, without requiring Dashboard interaction.

Additional resources

- [Enabling limited-support serving runtimes via Dashboard](#)
- [Enabling limited-support serving runtimes via GitOps](#)
- [Model-serving runtimes](#)
- [KServe Serving Runtimes](#)
- [LLMInferenceService Configuration](#)

1.5. MODEL-SERVING RUNTIMES FOR ACCELERATORS

OpenShift AI provides support for accelerators through preinstalled model-serving runtimes.

1.5.1. NVIDIA GPUs

You can serve models with NVIDIA graphics processing units (GPUs) by using the **vLLM NVIDIA GPU ServingRuntime for KServe** runtime. To use the runtime, you must enable GPU support in OpenShift AI. This includes installing and configuring the Node Feature Discovery Operator on your cluster. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).

1.5.2. Intel Gaudi accelerators

You can serve models with Intel Gaudi accelerators by using the **vLLM Intel Gaudi Accelerator ServingRuntime for KServe** runtime. To use the runtime, you must enable hybrid processing (HPU) support in OpenShift AI. This includes installing the Intel Gaudi Base Operator and configuring a hardware profile. For more information, see [Intel Gaudi Base Operator OpenShift installation](#) and [Working with hardware profiles](#).

For information about recommended vLLM parameters, environment variables, supported configurations and more, see [vLLM with Intel® Gaudi® AI Accelerators](#).



NOTE

Warm-up is a model initialization and performance optimization step that is useful for reducing cold-start delays and first-inference latency. Depending on the model size, warm-up can lead to longer model loading times.

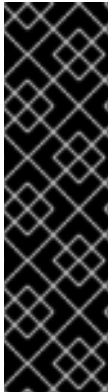
While highly recommended in production environments to avoid performance limitations, you can choose to skip warm-up for non-production environments to reduce model loading times and accelerate model development and testing cycles. To skip warm-up, follow the steps described in [Customizing the parameters of a deployed model-serving runtime](#) to add the following environment variable in the **Configuration parameters** section of your model deployment:

```
`VLLM_SKIP_WARMUP="true"
```

1.5.3. AMD GPUs

You can serve models with AMD GPUs by using the **vLLM AMD GPU ServingRuntime for KServe** runtime. To use the runtime, you must enable support for AMD graphic processing units (GPUs) in OpenShift AI. This includes installing the AMD GPU operator and configuring a hardware profile. For more information, see [Deploying the AMD GPU operator on OpenShift](#) in the AMD documentation and [Working with hardware profiles](#).

1.5.4. IBM Spyre AI accelerators on x86_64, IBM Power, and IBM Z



IMPORTANT

Support for IBM Spyre AI Accelerators on x86_64 is currently available in Red Hat OpenShift AI 3.5 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

Support for IBM Spyre AI Accelerators on s390x is currently available in Red Hat OpenShift AI 3.0 as a General Availability (GA) feature.

Support for IBM Spyre AI Accelerators on IBM Power is currently available in Red Hat OpenShift AI 3.3 as a General Availability (GA) feature.

You can serve models with IBM Spyre AI accelerators on x86_64 by using the **vLLM Spyre AI Accelerator ServingRuntime for KServe** runtime. For IBM Z (s390x architecture), use the **vLLM Spyre s390x ServingRuntime for KServe** runtime. For IBM Power (ppc64le architecture), use the **vLLM Spyre ppc64le ServingRuntime for KServe** runtime. To use the runtime, you must install the Spyre Operator and configure a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).

Additional resources

- [Supported model-serving runtimes](#)

1.5.5. Supported model-serving runtimes

OpenShift AI includes several preinstalled model-serving runtimes. You can use preinstalled model-serving runtimes to start serving models without modifying or defining the runtime yourself. You can also add a custom runtime to support a model.

See [Supported Configurations for 3.x](#) for a list of the supported model-serving runtimes and deployment requirements.

For help adding a custom runtime, see [Adding a custom model-serving runtime](#).

Additional resources

- [Inference endpoints](#)

1.5.6. Tested and verified model-serving runtimes

Tested and verified runtimes are community versions of model-serving runtimes that have been tested and verified against specific versions of OpenShift AI.

Red Hat tests the current version of a tested and verified runtime each time there is a new version of OpenShift AI. If a new version of a tested and verified runtime is released in the middle of an OpenShift AI release cycle, it will be tested and verified in an upcoming release.

See [Supported Configurations for 3.x](#) for a list of tested and verified runtimes in OpenShift AI.



NOTE

Tested and verified runtimes are not directly supported by Red Hat. You are responsible for ensuring that you are licensed to use any tested and verified runtimes that you add, and for correctly configuring and maintaining them.

For more information, see [Tested and verified runtimes in OpenShift AI](#) .

Additional resources

- [Inference endpoints](#)

CHAPTER 2. CONFIGURE MODEL SERVERS

You configure model servers by using model-serving runtimes, which add support for a specified set of model frameworks and the model formats that they support.

2.1. ENABLE THE MODEL SERVING PLATFORM

When you have installed KServe, you can use the Red Hat OpenShift AI dashboard to enable the model serving platform. You can also use the dashboard to enable model-serving runtimes for the platform.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have installed KServe.

Procedure

1. Enable the model serving platform as follows:
 - a. In the left menu, click **Settings** → **Cluster settings** → **General settings**.
 - b. Locate the **Model serving platforms** section.
 - c. To enable the model serving platform for projects, select the **Model serving platform** checkbox.
 - d. Click **Save changes**.
2. Enable preinstalled runtimes for the model serving platform as follows:
 - a. In the left menu of the OpenShift AI dashboard, click **Settings** → **Model resources and operations** → **Serving runtimes**.
The **Serving runtimes** page shows preinstalled runtimes and any custom runtimes that you have added.

For more information about preinstalled runtimes, see [Supported runtimes](#).
 - b. Set the runtime that you want to use to **Enabled**.
The model serving platform is now available for model deployments.

2.2. ENABLE SPECULATIVE DECODING AND MULTI-MODAL INFERENCE

You can configure the **vLLM NVIDIA GPU ServingRuntime for KServer** runtime to use speculative decoding, a parallel processing technique to optimize inferencing time for large language models (LLMs).

You can also configure the runtime to support inferencing for vision-language models (VLMs). VLMs are a subset of multi-modal models that integrate both visual and textual data.

The following procedure describes customizing the **vLLM NVIDIA GPU ServingRuntime for KServe** runtime for speculative decoding and multi-modal inferencing.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- If you are using the vLLM model-serving runtime for speculative decoding with a draft model, you have stored the original model and the speculative model in the same folder within your S3-compatible object storage.

Procedure

1. Follow the steps to deploy a model as described in [Deploying models on the model serving platform](#).
2. In the **Serving runtime** field, select the **vLLM NVIDIA GPU ServingRuntime for KServe** runtime.
3. To configure the vLLM model-serving runtime for speculative decoding by matching n-grams in the prompt, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```
--speculative-model=[ngram]
--num-speculative-tokens=<NUM_SPECULATIVE_TOKENS>
--ngram-prompt-lookup-max=<NGRAM_PROMPT_LOOKUP_MAX>
--use-v2-block-manager
```

- a. Replace **<NUM_SPECULATIVE_TOKENS>** and **<NGRAM_PROMPT_LOOKUP_MAX>** with your own values.



NOTE

Inferencing throughput varies depending on the model used for speculating with n-grams.

4. To configure the vLLM model-serving runtime for speculative decoding with a draft model, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```
--port=8080
--served-model-name={{.Name}}
--distributed-executor-backend=mp
--model=/mnt/models/<path_to_original_model>
--speculative-model=/mnt/models/<path_to_speculative_model>
--num-speculative-tokens=<NUM_SPECULATIVE_TOKENS>
--use-v2-block-manager
```

- a. Replace **<path_to_speculative_model>** and **<path_to_original_model>** with the paths to the speculative model and original model on your S3-compatible object storage.
 - b. Replace **<NUM_SPECULATIVE_TOKENS>** with your own value.
5. To configure the vLLM model-serving runtime for multi-modal inferencing, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```
--trust-remote-code
```

**NOTE**

Only use the **--trust-remote-code** argument with models from trusted sources.

6. Click **Deploy**.

Verification

- If you have configured the vLLM model-serving runtime for speculative decoding, use the following example command to verify API requests to your deployed model:

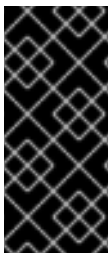
```
curl -v https://<inference_endpoint_url>:443/v1/chat/completions
-H "Content-Type: application/json"
-H "Authorization: Bearer <token>"
```

- If you have configured the vLLM model-serving runtime for multi-modal inferencing, use the following example command to verify API requests to the vision-language model (VLM) that you have deployed:

```
curl -v https://<inference_endpoint_url>:443/v1/chat/completions
-H "Content-Type: application/json"
-H "Authorization: Bearer <token>"
-d '{"model": "<model_name>",
  "messages":
    [{"role": "<role>",
      "content":
        [{"type": "text", "text": "<text>"
        },
        {"type": "image_url", "image_url": "<image_url_link>"
        }
      ]
    }
  ]
}'
```

2.3. ADDING A CUSTOM MODEL-SERVING RUNTIME

Add custom model-serving runtimes to support model frameworks and formats that are not available in preinstalled runtimes, enabling you to deploy models that require specialized serving infrastructure.

**IMPORTANT**

Custom runtimes are user-created resources and are distinct from limited-support runtimes. Custom runtimes receive no Red Hat support coverage, while limited-support runtimes (such as vLLM fast builds) are Red Hat-built with a defined 1-month support window. For information about enabling limited-support runtimes, see [Enabling limited-support serving runtimes via Dashboard](#).

**NOTE**

Red Hat does not provide support for custom runtimes. You are responsible for ensuring that you are licensed to use any custom runtimes that you add, and for correctly configuring and maintaining them.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have built your custom runtime and added the image to a container image repository such as [Quay](#).

Procedure

1. From the OpenShift AI dashboard, click **Settings → Model resources and operations → Serving runtimes**.
The **Serving runtimes** page opens and shows the model-serving runtimes that are already installed and enabled.
2. To add a custom runtime, choose one of the following options:
 - To start with an existing runtime (for example, **vLLM NVIDIA GPU ServingRuntime for KServe**), click the action menu (**:**) next to the existing runtime and then click **Duplicate**.
 - To add a new custom runtime, click **Add serving runtime**.
3. In the **Select the model serving platforms this runtime supports** list, select **Single-model serving platform**.
4. In the **Select the API protocol this runtime supports** list, select **REST** or **gRPC**.
5. Optional: If you started a new runtime (rather than duplicating an existing one), add your code by choosing one of the following options:
 - **Upload a YAML file**
 - a. Click **Upload files**.
 - b. In the file browser, select a YAML file on your computer.
The embedded YAML editor opens and shows the contents of the file that you uploaded.
 - **Enter YAML code directly in the editor**
 - a. Click **Start from scratch**
 - b. Enter or paste YAML code directly in the embedded editor.

**NOTE**

In many cases, creating a custom runtime will require adding new or custom parameters to the **env** section of the **ServingRuntime** specification.

6. Click **Add**.

The **Serving runtimes** page opens and shows the updated list of runtimes that are installed. Observe that the custom runtime that you added is automatically enabled. The API protocol that you specified when creating the runtime is shown.

7. Optional: To edit your custom runtime, click the action menu (⋮) and select **Edit**.

Verification

- The custom model-serving runtime that you added is shown in an enabled state on the **Serving runtimes** page.

2.4. ADD A TESTED AND VERIFIED RUNTIME

In addition to preinstalled and custom model-serving runtimes, you can also use Red Hat tested and verified model-serving runtimes to support your requirements. For more information about Red Hat tested and verified runtimes, see [Tested and verified runtimes for Red Hat OpenShift AI](#).

You can use the Red Hat OpenShift AI dashboard to add and enable tested and verified runtimes for the model serving platform. You can then choose the runtime when you deploy a model on the model serving platform.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- If you are deploying the IBM Z Accelerated for NVIDIA Triton Inference Server runtime, you have access to IBM Cloud Container Registry to pull the container image. For more information about obtaining credentials to the IBM Cloud Container Registry, see [Downloading the IBM Z Accelerated for NVIDIA Triton Inference Server container image](#).
- If you are deploying the IBM Power Accelerated Triton Inference Server runtime, you can access the container image from the [Triton Inference Server Quay repository](#).

Procedure

1. From the OpenShift AI dashboard, click **Settings → Model resources and operations → Serving runtimes**.
The **Serving runtimes** page opens and shows the model-serving runtimes that are already installed and enabled.
2. Click **Add serving runtime**.
3. In the **Select the model serving platforms this runtime supports** list, select **Single-model serving platform**.
4. In the **Select the API protocol this runtime supports** list, select **REST** or **gRPC**.
5. Click **Start from scratch**.
6. Follow these steps to add the **IBM Power Accelerated for NVIDIA Triton Inference Server** runtime:
 - a. If you selected the **REST** API protocol, enter or paste the following YAML code directly in the embedded editor.

```
apiVersion: serving.kserve.io/v1alpha1
```

```

kind: ServingRuntime
metadata:
  name: triton-ppc64le-runtime
  annotations:
    openshift.io/display-name: Triton Server ServingRuntime for KServe(ppc64le)
spec:
  supportedModelFormats:
    - name: FIL
      version: "1"
      autoSelect: true
    - name: python
      version: "1"
      autoSelect: true
    - name: onnx
      version: "1"
      autoSelect: true
    - name: pytorch
      version: "1"
      autoSelect: true
  multiModel: false
  containers:
    - command:
      - tritonserver
      - --model-repository=/mnt/models
      name: kserve-container
      image: quay.io/powercloud/tritonserver:latest
      resources:
        requests:
          cpu: 2
          memory: 8Gi
        limits:
          cpu: 2
          memory: 8Gi
      ports:
        - containerPort: 8000

```

7. Follow these steps to add the **IBM Z Accelerated for NVIDIA Triton Inference Server** runtime:
 - a. If you selected the **REST** API protocol, enter or paste the following YAML code directly in the embedded editor.

```

apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: ibmz-triton-rest
  labels:
    opendatahub.io/dashboard: "true"
spec:
  containers:
    - name: kserve-container
      command:
        - /bin/sh
        - -c
      args:
        - /opt/tritonserver/bin/tritonserver --model-repository=/mnt/models --http-port=8000 -
          -grpc-port=8001 --metrics-port=8002

```

```

image: icr.io/ibmz/ibmz-accelerated-for-nvidia-triton-inference-server:<version>
securityContext:
  allowPrivilegeEscalation: false
  capabilities:
    drop:
      - ALL
  runAsNonRoot: true
  seccompProfile:
    type: RuntimeDefault
resources:
  limits:
    cpu: "2"
    memory: 4Gi
  requests:
    cpu: "2"
    memory: 4Gi
ports:
  - containerPort: 8000
    protocol: TCP
protocolVersions:
  - v2
  - grpc-v2
supportedModelFormats:
  - name: onnx-mlir
    version: "1"
    autoSelect: true
  - name: snapml
    version: "1"
    autoSelect: true
  - name: pytorch
    version: "1"
    autoSelect: true

```

- b. If you selected the **gRPC** API protocol, enter or paste the following YAML code directly in the embedded editor.

```

apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: ibmz-triton-grpc
  labels:
    opendatahub.io/dashboard: "true"
spec:
  containers:
    - name: kserve-container
      command:
        - /bin/sh
        - -c
      args:
        - /opt/tritonserver/bin/tritonserver --model-repository=/mnt/models --grpc-port=8001
        --http-port=8000 --metrics-port=8002
      image: icr.io/ibmz/ibmz-accelerated-for-nvidia-triton-inference-server:<version>
      securityContext:
        allowPrivilegeEscalation: false
        capabilities:
          drop:

```

```

- ALL
runAsNonRoot: true
seccompProfile:
  type: RuntimeDefault
resources:
  limits:
    cpu: "2"
    memory: 4Gi
  requests:
    cpu: "2"
    memory: 4Gi
ports:
- containerPort: 8001
  name: grpc
  protocol: TCP
volumeMounts:
- mountPath: /dev/shm
  name: shm
protocolVersions:
- v2
- grpc-v2
supportedModelFormats:
- name: onnx-mlir
  version: "1"
  autoSelect: true
- name: snapml
  version: "1"
  autoSelect: true
- name: pytorch
  version: "1"
  autoSelect: true
volumes:
- emptyDir: null
  medium: Memory
  sizeLimit: 2Gi
  name: shm

```

8. Follow these steps to add the **NVIDIA Triton Inference Server** runtime:

- a. If you selected the **REST** API protocol, enter or paste the following YAML code directly in the embedded editor.

```

apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: triton-kserve-rest
  labels:
    opendatahub.io/dashboard: "true"
spec:
  annotations:
    prometheus.kserve.io/path: /metrics
    prometheus.kserve.io/port: "8002"
  containers:
  - args:
    - tritonserver
    - --model-store=/mnt/models

```

```

- --grpc-port=9000
- --http-port=8080
- --allow-grpc=true
- --allow-http=true
image: nvcr.io/nvidia/tritonserver@sha256:xxxxx
name: kserve-container
resources:
  limits:
    cpu: "1"
    memory: 2Gi
  requests:
    cpu: "1"
    memory: 2Gi
  ports:
    - containerPort: 8080
      protocol: TCP
protocolVersions:
- v2
- grpc-v2
supportedModelFormats:
- autoSelect: true
  name: tensorrt
  version: "8"
- autoSelect: true
  name: tensorflow
  version: "1"
- autoSelect: true
  name: tensorflow
  version: "2"
- autoSelect: true
  name: onnx
  version: "1"
- name: pytorch
  version: "1"
- autoSelect: true
  name: triton
  version: "2"
- autoSelect: true
  name: xgboost
  version: "1"
- autoSelect: true
  name: python
  version: "1"

```

- b. If you selected the **gRPC** API protocol, enter or paste the following YAML code directly in the embedded editor.

```

apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: triton-kserve-grpc
labels:
  opendatahub.io/dashboard: "true"
spec:
  annotations:
    prometheus.kserve.io/path: /metrics

```

```
prometheus.kserve.io/port: "8002"
containers:
  - args:
    - tritonserver
    - --model-store=/mnt/models
    - --grpc-port=9000
    - --http-port=8080
    - --allow-grpc=true
    - --allow-http=true
  image: nvcr.io/nvidia/tritonserver@sha256:xxxxx
  name: kserve-container
  ports:
    - containerPort: 9000
      name: h2c
      protocol: TCP
  volumeMounts:
    - mountPath: /dev/shm
      name: shm
  resources:
    limits:
      cpu: "1"
      memory: 2Gi
    requests:
      cpu: "1"
      memory: 2Gi
  protocolVersions:
    - v2
    - grpc-v2
  supportedModelFormats:
    - autoSelect: true
      name: tensorrt
      version: "8"
    - autoSelect: true
      name: tensorflow
      version: "1"
    - autoSelect: true
      name: tensorflow
      version: "2"
    - autoSelect: true
      name: onnx
      version: "1"
    - name: pytorch
      version: "1"
    - autoSelect: true
      name: triton
      version: "2"
    - autoSelect: true
      name: xgboost
      version: "1"
    - autoSelect: true
      name: python
      version: "1"
  volumes:
    - name: shm
```

```
emptyDir: null
medium: Memory
sizeLimit: 2Gi
```

9. In the **metadata.name** field, make sure that the value of the runtime you are adding does not match a runtime that you have already added.
10. Optional: To use a custom display name for the runtime that you are adding, add a **metadata.annotations.openshift.io/display-name** field and specify a value, as shown in the following example:

```
apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: kserve-triton
  annotations:
    openshift.io/display-name: Triton ServingRuntime
```



NOTE

If you do not configure a custom display name for your runtime, OpenShift AI shows the value of the **metadata.name** field.

11. Click **Create**.
The **Serving runtimes** page opens and shows the updated list of runtimes that are installed. Observe that the runtime that you added is automatically enabled. The API protocol that you specified when creating the runtime is shown.
12. Optional: To edit the runtime, click the action menu (⋮) and select **Edit**.

Verification

- The model-serving runtime that you added is shown in an enabled state on the **Serving runtimes** page.

Additional resources

- [Tested and verified model-serving runtimes](#)

2.5. ENABLE LIMITED-SUPPORT SERVING RUNTIMES FROM THE DASHBOARD

As an administrator, you can enable Red Hat-built limited-support vLLM fast-build runtimes, where available in your version of OpenShift AI, through the Dashboard by accepting a risk acknowledgment modal.

Limited-support vLLM runtimes and accelerator configurations are hidden from data scientists until you explicitly accept the risk and enable them. The Dashboard workflow guides you through understanding the 1-month support window and accepting responsibility for runtime updates.

Prerequisites

- You have logged in to OpenShift AI with administrator privileges.

- You have access to **Settings → Model resources and operations → Serving runtimes** in the OpenShift AI Dashboard.
- You understand the 1-month support coverage window for limited-support runtimes.
- If you plan to enable limited-support vLLM accelerator configurations (LLMInferenceServiceConfigs), your cluster administrator has enabled the vLLM deployment on MaaS feature flag: **spec.dashboardConfig.vLLMDeploymentOnMaaS: true** in the **OdhDashboardConfig** custom resource. This feature is available as a Technology Preview.

Procedure

1. In the OpenShift AI Dashboard, click **Settings → Model resources and operations** and choose one of the following:
 - **Serving runtimes** - To enable limited-support ServingRuntime templates
 - **LLM accelerator configurations** - To enable limited-support LLMInferenceServiceConfig accelerator configurations (requires the **vLLMDeploymentOnMaaS** feature flag enabled)
The selected page opens and shows the available runtimes or configurations.
2. Scroll to identify vLLM runtimes marked with the **Limited support** badge and **fast-N** version badge.
For example, "vLLM ServingRuntime for KServe" with badges "Limited support", "0.6.1", and "fast-1".
3. Locate the toggle switch for the limited-support runtime you want to enable.
4. Click the toggle switch to enable the runtime or configuration.
A modal is displayed with text specific to the resource type:
 - For serving runtimes: **Enable limited-support runtime?** with the message "The support coverage for this runtime is limited to 1 month after release."
 - For accelerator configurations: **Enable limited-support accelerator configuration?** with the message "The support coverage for this accelerator configuration is limited to 1 month after release."
5. Select the **I understand** checkbox.
6. Click **Enable**.
The modal closes. The runtime now is displayed as enabled on the **Serving runtimes** page with the **Limited support** badge.
7. Optional: If you manage runtimes or configurations via GitOps, verify that the acceptance annotation was set:

```
# For serving runtimes:
$ oc get template <runtime-name> -o yaml | grep unsupported-status-accepted

# For accelerator configurations:
$ oc get llminferenceserviceconfig <config-name> -o yaml | grep unsupported-status-accepted
```

Example output

```
opendatahub.io/unsupported-status-accepted: "true"
```

Verification

- The limited-support vLLM runtime is displayed in an enabled state on the **Serving runtimes** page with the **Limited support** badge.
- Data scientists can now see the runtime in the model deployment wizard runtime selection dropdown.
- The runtime displays three badges: "Limited support", version, and "fast-N".

Additional resources

- [Understanding vLLM runtime support levels](#)
- [Enabling limited-support serving runtimes via GitOps](#)
- [Limited-support runtime state and backward compatibility](#)

2.6. ENABLE LIMITED-SUPPORT SERVING RUNTIMES USING GITOPS

You can enable Red Hat-built limited-support vLLM runtimes and accelerator configurations, where available in your version of OpenShift AI, declaratively by adding acceptance annotations to **ServingRuntime** or **LLMInferenceServiceConfig** YAML manifests in your GitOps pipeline. This approach does not require Dashboard interaction while maintaining auditable acceptance records in Git history.

Prerequisites

- You have a GitOps pipeline (for example, ArgoCD, Flux) configured with cluster-admin RBAC permissions.
- You have access to the **ServingRuntime** or **LLMInferenceServiceConfig** YAML templates for the fast-build vLLM runtime you want to enable.
- You understand the 1-month support coverage window for limited-support runtimes.
- You have documented your acceptance decision in your change management process.

Procedure

1. Obtain the **ServingRuntime** or **LLMInferenceServiceConfig** YAML template for the limited-support vLLM runtime.
2. Add the required annotations to the **metadata.annotations** section. For example, a fast-1 vLLM runtime template:

ServingRuntime fast-1 vLLM runtime template example

```
apiVersion: serving.kserve.io/v1alpha1
kind: ServingRuntime
metadata:
  name: vllm-runtime-fast-1
```

```

annotations:
  opendatahub.io/support-status: unsupported
  opendatahub.io/unsupported-status-accepted: "true"
  opendatahub.io/fast-version: "1"
  opendatahub.io/recommended-accelerators: ["nvidia.com/gpu"]
  openshift.io/display-name: vLLM ServingRuntime for KServe (fast-1)
spec:
  # ... (runtime specification)

```

Where:

- **opendatahub.io/support-status:** Marks the runtime as limited-support, requiring acceptance.
- **opendatahub.io/unsupported-status-accepted:** Indicates administrator acceptance of the 1-month support window.
- **opendatahub.io/fast-version:** Fast build version number. The Dashboard displays this with a "fast-" prefix (for example, a value of "1" displays as "fast-1").

LLMInferenceServiceConfig example

```

apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceServiceConfig
metadata:
  name: vllm-config-fast-1
  annotations:
    opendatahub.io/support-status: unsupported
    opendatahub.io/unsupported-status-accepted: "true"
    opendatahub.io/fast-version: "1"
spec:
  # ... (configuration specification)

```

- **opendatahub.io/support-status:** Marks the configuration as limited-support.
 - **opendatahub.io/unsupported-status-accepted:** Indicates administrator acceptance via GitOps.
 - **opendatahub.io/fast-version:** Fast build version number. The Dashboard displays this with a "fast-" prefix (for example, a value of "1" displays as "fast-1").
3. Commit the YAML manifest to your GitOps repository with a descriptive commit message documenting your acceptance decision:

```

$ git add runtimes/vllm-runtime-fast-1.yaml
$ git commit -m "Enable vLLM fast-1 runtime for experimentation workloads"

```

```

Accepted limited-support status with 1-month support window.
Migration to fast-2 or GA runtime planned before support expiration.
Risk accepted by: Admin Name <admin@example.com>
JIRA: RHOAI-1234"

```

4. Push the commit to trigger your GitOps sync:

```

$ git push origin main

```

5. Wait for your GitOps tool to sync the changes to the cluster.
6. Verify that the runtime or configuration is displayed in the OpenShift AI Dashboard:
 - a. Go to **Settings → Model resources and operations → Serving runtimes** (for ServingRuntimes) or **Settings → Model resources and operations → LLM accelerator configurations** (for LLMInferenceServiceConfigs).
 - b. Confirm that the limited-support runtime or configuration displays with the **Limited support** badge.
7. Verify that the annotations are correctly applied:

```
# For serving runtimes:
$ oc get template vllm-runtime-fast-1 -o yaml | grep -A 2 "opendatahub.io/support-status"

# For accelerator configurations:
$ oc get llminferenceserviceconfig vllm-config-fast-1 -o yaml | grep -A 2
"opendatahub.io/support-status"
```

Example output

```
opendatahub.io/support-status: unsupported
opendatahub.io/unsupported-status-accepted: "true"
opendatahub.io/fast-version: "1"
```

Verification

- The limited-support vLLM runtime is displayed in the OpenShift AI Dashboard with the **Limited support** badge.
- Data scientists can see the runtime in model deployment workflows.
- The **opendatahub.io/unsupported-status-accepted** annotation is set to **"true"** in the cluster resource.
- Your Git commit history provides an auditable record of the acceptance decision.

Additional resources

- [Understanding vLLM runtime support levels](#)
- [Understanding limited-support runtime visibility and gating](#)

2.7. LIMITED-SUPPORT RUNTIME STATE AND BACKWARD COMPATIBILITY

The acceptance annotation persists when you disable and re-enable limited-support runtimes, preventing repeated risk acknowledgment prompts. Existing custom runtimes without support status annotations continue to work without modification.

OpenShift AI manages limited-support runtime state through persistent annotations and backward-compatible defaults. Understanding state persistence and backward compatibility helps you support runtimes across enable/disable cycles and upgrade to new OpenShift AI versions.

Acceptance state persistence

When you accept a limited-support runtime by using the Dashboard or GitOps, the **opendatahub.io/unsupported-status-accepted** annotation is set to **"true"** on the resource. This annotation persists through the following operations:

- Disabling the runtime via the Dashboard toggle
- Re-enabling the runtime via the Dashboard toggle
- OpenShift AI operator upgrades
- Cluster maintenance and restarts

The acceptance lifecycle works as follows:

1. **Initial enable:** You toggle the runtime to enabled. The risk acceptance modal is displayed.
2. **Accept risk:** You check "I understand" and click **Enable**. The annotation **opendatahub.io/unsupported-status-accepted: "true"** is added.
3. **Disable runtime:** You toggle the runtime to disabled. The acceptance annotation remains **"true"**.
4. **Re-enable runtime:** You toggle the runtime back to enabled. No modal is displayed because the annotation is still **"true"**.

This persistence reduces administrative burden and avoids repeated acceptance workflows for temporarily disabled runtimes.

Lifecycle management considerations

When managing limited-support runtimes over time, consider:

Disabling for maintenance

If you disable a limited-support runtime for cluster maintenance or troubleshooting, you can re-enable it without re-accepting the risk modal.

Support window expiration

The acceptance annotation does not automatically expire when the 1-month support window ends. You remain responsible for tracking support expiration and planning migrations to newer fast builds or GA runtimes.

Removing acceptance

To revoke acceptance and hide the runtime, remove the **opendatahub.io/unsupported-status-accepted** annotation or set it to **"false"**:

```
$ oc annotate servingruntime <runtime-name> \
  opendatahub.io/unsupported-status-accepted-
```

Backward compatibility for existing custom runtimes

The support status annotations are purely additive. Runtimes without the **opendatahub.io/support-status** annotation are treated as supported (visible) by default.

This ensures backward compatibility for:

Custom runtimes created before OpenShift AI 3.5

Existing custom ServingRuntime and LLMInferenceServiceConfig resources without support status annotations continue to function and remain visible. No changes are required to maintain existing runtimes.

User-created runtimes

When you add a new custom runtime via the Dashboard or GitOps, do not include the **opendatahub.io/support-status** annotation unless you specifically want to mark it as unsupported and require acceptance.

Default visibility behavior

Absence of the **support-status** annotation is equivalent to **support-status: supported**. The runtime is visible without requiring acceptance.

When to add annotations to custom runtimes

Add the support status annotations to a custom runtime only when:

- You want to mark a user-created runtime as unsupported and require administrator acceptance before it becomes visible.
- You are building an internal fast-build process and want to apply the same gating workflow as Red Hat-provided fast builds.
- You have organizational policies requiring risk acceptance for experimental or community-maintained runtimes.

For standard custom runtimes that should be immediately visible, omit the annotations entirely.

GitOps state management

When managing limited-support runtimes via GitOps:

- The acceptance annotation is declarative and stored in your Git repository.
- Git commit history provides an auditable trail of acceptance decisions.
- Removing the annotation from your manifest and syncing will revert the runtime to a hidden state.
- Adding the annotation back and syncing will make the runtime visible again without Dashboard interaction.

Example of revoking acceptance via GitOps:

Before (accepted state)

```
metadata:
  annotations:
    opendatahub.io/support-status: unsupported
    opendatahub.io/unsupported-status-accepted: "true"
```

After (revoked state)

```
metadata:
  annotations:
```

```
opendatahub.io/support-status: unsupported
# unsupported-status-accepted annotation removed
```

After syncing, the runtime becomes hidden from data scientists.

Additional resources

- [Understanding limited-support runtime visibility and gating](#)
- [Adding a custom model-serving runtime](#)

2.8. ENABLE THE NVIDIA NIM MODEL SERVING PLATFORM

As an OpenShift AI administrator, you can use the Red Hat OpenShift AI dashboard to enable the NVIDIA NIM model serving platform.



NOTE

If you previously enabled the **NVIDIA NIM model serving platform** in OpenShift AI, and then upgraded to a newer version, re-enter your NVIDIA personal API key to re-enable the NVIDIA NIM model serving platform.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have enabled the model serving platform. You do not need to enable a preinstalled runtime. For more information about enabling the model serving platform, see [Enabling the model serving platform](#).
- The **disableNIMModelServing** dashboard configuration option is set to **false**. For more information about setting dashboard configuration options, see [Customizing the dashboard](#).
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have an NVIDIA Cloud Account (NCA) and can access the NVIDIA GPU Cloud (NGC) portal. For more information, see [NVIDIA GPU Cloud user guide](#).
- Your NCA account is associated with the NVIDIA AI Enterprise Viewer role.
- You have generated a personal API key on the NGC portal. For more information, see [Generating a Personal API Key](#).

Procedure

1. In the left menu of the OpenShift AI dashboard, click **Applications → Explore**.
2. On the **Explore** page, find the **NVIDIA NIM** tile.
3. Click **Enable** on the application tile.
4. Enter your personal API key and then click **Submit**.

Verification

- The NVIDIA NIM application that you enabled is displayed on the **Enabled** page.

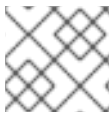
CHAPTER 3. CUSTOMIZE MODEL DEPLOYMENTS

You can customize a model's deployment to suit your specific needs, for example, to deploy a particular family of models or to enhance an existing deployment. You can modify the runtime configuration for a specific deployment by setting additional serving runtime arguments and environment variables.

These customizations apply only to the selected model deployment and do not change the default runtime configuration. You can set these parameters when you first deploy a model or by editing an existing deployment.

3.1. CUSTOMIZE THE PARAMETERS OF A DEPLOYED MODEL-SERVING RUNTIME

You might need additional parameters beyond the default ones to deploy specific models or to enhance an existing model deployment. In such cases, you can modify the parameters of an existing runtime to suit your deployment needs.



NOTE

Customizing the parameters of a runtime only affects the selected model deployment.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have deployed a model.

Procedure

1. From the OpenShift AI dashboard, click **AI hub** → **Deployments**.
The **Deployments** page opens.
2. Click **Stop** next to the name of the model you want to customize.
3. Click the action menu () and select **Edit**.
The **Configuration parameters** section shows predefined serving runtime parameters, if any are available.
4. Customize the runtime parameters in the **Configuration parameters** section:
 - a. Modify the values in **Additional serving runtime arguments** to define how the deployed model behaves.
 - b. Modify the values in **Additional environment variables** to define variables in the model's environment.



NOTE

Do not modify the port or model serving runtime arguments, because they require specific values to be set. Overwriting these parameters can cause the deployment to fail.

**NOTE**

Set **VLLM_CPU_KVCACHE_SPACE** to define the KV cache size for vLLM. For example, **VLLM_CPU_KVCACHE_SPACE=40** allocates 40 GiB of memory to the KV cache. Increase this value to enable vLLM to handle more parallel requests. Choose a value that matches your hardware capacity and memory management requirements. The default is 0. When set to 0, vLLM does not reserve dedicated KV cache memory and instead allocates from available system memory at runtime, which can result in out-of-memory errors.

5. After you are done customizing the runtime parameters, click **Redeploy** to save.
6. Click **Start** to deploy the model with your changes.

Verification

- Confirm that the deployed model is shown on the **Deployments** tab for the project, and on the **Deployments** page of the dashboard with a checkmark in the **Status** column.
- Confirm that the arguments and variables that you set appear in **spec.predictor.model.args** and **spec.predictor.model.env** by one of the following methods:
 - Checking the InferenceService YAML from the OpenShift Console.
 - Using the following command in the OpenShift CLI:

```
oc get -o json inferencesservice <inferencesservicename/modelname> -n <projectname>
```

3.2. CUSTOMIZABLE MODEL SERVING RUNTIME PARAMETERS

You can modify the parameters of an existing model serving runtime to suit your deployment needs.

For more information about parameters for each of the supported serving runtimes, see the following table:

Serving runtime	Resource
NVIDIA Triton Inference Server	NVIDIA Triton Inference Server: Model Parameters
OpenVINO Model Server	OpenVINO Model Server Features: Dynamic Input Parameters
vLLM NVIDIA GPU ServingRuntime for KServe	vLLM: Engine Arguments OpenAI-Compatible Server
vLLM AMD GPU ServingRuntime for KServe	vLLM: Engine Arguments OpenAI-Compatible Server
vLLM Intel Gaudi Accelerator ServingRuntime for KServe	vLLM: Engine Arguments OpenAI-Compatible Server

Serving runtime	Resource
vLLM Spyre ppc64le ServingRuntime for KServe	Recommended model inference settings for IBM Power with IBM Spyre AI accelerators

Additional resources

- [Customizing the parameters of a deployed model serving runtime](#)

3.3. CUSTOMIZE THE VLLM MODEL-SERVING RUNTIME

In certain cases, you may need to add additional flags or environment variables to the **vLLM ServingRuntime for KServe** runtime to deploy a family of LLMs.

The following procedure describes customizing the vLLM model-serving runtime to deploy a Llama, Granite or Mistral model.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- For Llama model deployment, you have downloaded a meta-llama-3 model to your object storage.
- For Granite model deployment, you have downloaded a granite-7b-instruct or granite-20B-code-instruct model to your object storage.
- For Mistral model deployment, you have downloaded a mistral-7B-Instruct-v0.3 model to your object storage.
- You have enabled the **vLLM ServingRuntime for KServeruntime**.
- You have enabled GPU support in OpenShift AI and have installed and configured the Node Feature Discovery Operator on your cluster. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#)

Procedure

1. Follow the steps to deploy a model as described in [Deploying models on the model serving platform](#).
2. In the **Serving runtime** field, select **vLLM ServingRuntime for KServe**
3. If you are deploying a meta-llama-3 model, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```

--distributed-executor-backend=mp
--max-model-len=6144

```

Where:

- **--distributed-executor-backend=mp**: Sets the backend to multiprocessing for distributed model workers

- **--max-model-len=6144**: Sets the maximum context length of the model to 6144 tokens

4. If you are deploying a granite-7B-instruct model, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```
--distributed-executor-backend=mp
```

Where:

- **--distributed-executor-backend=mp**: Sets the backend to multiprocessing for distributed model workers

5. If you are deploying a granite-20B-code-instruct model, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```
--distributed-executor-backend=mp
--tensor-parallel-size=4
--max-model-len=6448
```

Where:

- **--distributed-executor-backend=mp**: Sets the backend to multiprocessing for distributed model workers
- **--tensor-parallel-size=4**: Distributes inference across 4 GPUs in a single node
- **--max-model-len=6448**: Sets the maximum context length of the model to 6448 tokens

6. If you are deploying a mistral-7B-Instruct-v0.3 model, add the following arguments under **Additional serving runtime arguments** in the **Configuration parameters** section:

```
--distributed-executor-backend=mp
--max-model-len=15344
```

Where:

- **--distributed-executor-backend=mp**: Sets the backend to multiprocessing for distributed model workers
- **--max-model-len=15344**: Sets the maximum context length of the model to 15344 tokens

7. Click **Deploy**.

Verification

- Confirm that the deployed model is shown on the **Models** tab for the project, and on the **Model deployments** page of the dashboard with a checkmark in the **Status** column.
- For granite models, use the following example command to verify API requests to your deployed model:

```
curl -q -X 'POST' \
  "https://<inference_endpoint_url>:443/v1/chat/completions" \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
```

```
-d "{
  \"model\": \"<model_name>\",
  \"prompt\": \"<prompt>\",
  \"max_tokens\": <max_tokens>,
  \"temperature\": <temperature>
}"
```

Additional resources

- [vLLM: Engine Arguments](#)

3.4. SET A DEFAULT CLUSTER-WIDE DEPLOYMENT STRATEGY

You can set a default deployment strategy for new model deployments across the cluster.

Prerequisites

- You have logged in to OpenShift AI as a user with OpenShift AI administrator privileges.
- You have enabled model serving on your cluster.

Procedure

1. In the dashboard, navigate to **Settings → Cluster settings**.
2. Click on the **General settings** tab.
3. Scroll down to the **Model deployment options** section.
4. In the **Default deployment strategy**, select the desired cluster default:
 - **Rolling update**
 - **Recreate**
5. Click **Save changes** at the bottom of the page.

Verification

- Follow the instructions to deploy a new model as described in [Deploying models on the model serving platform](#).
- In the **Advanced settings** page of the deployment wizard, locate the **Deployment strategy** section.
- The preselected deployment strategy should match the new default you configured.

CHAPTER 4. CACHE MODELS ON LOCAL STORAGE

You can pre-download and cache large language model artifacts on node-local Non-Volatile Memory Express (NVMe) storage to significantly reduce **InferenceService** cold start latency.

4.1. MODEL CACHE FOR FASTER INFERENCE STARTUP

You can use Model Cache to pre-download and store large language model (LLM) artifacts on node-local storage, such as Non-Volatile Memory Express (NVMe) volumes. Model Cache significantly reduces the time it takes for model serving pods to start, because pods mount locally cached model copies instead of downloading from remote storage on every startup. This makes autoscaling practical for latency-sensitive generative AI workloads, where new pods must begin serving requests quickly to handle demand spikes. Actual startup times depend on model size and storage performance.

4.1.1. Cold start latency and model caching

Cold start latency is the delay between when a model serving pod starts and when it is ready to serve inference requests. This delay occurs primarily because the pod must download the model from remote storage such as S3-compatible object storage or Hugging Face Hub before it can begin serving. For large models such as Llama 3 70B, this download can take up to 15-20 minutes or more depending on network bandwidth, which creates several problems:

- Autoscaling becomes impractical because new pods cannot serve traffic quickly enough to handle demand spikes.
- Scale-from-zero scenarios fail to meet latency requirements.
- GPU resources remain idle during the download period.
- Download timeouts or network failures can cause deployment failures.

Model Cache solves these problems by pre-downloading model artifacts to persistent local storage on designated nodes. When an **InferenceService** pod starts on a node that has a cached copy of the model, **KServe** detects the local copy and mounts it directly. The pod starts serving inference requests much faster than if it downloaded the full model from remote storage.

4.1.2. Custom resources

Model Cache uses four custom resources to manage cached models:

LocalModelCache

A cluster-scoped resource that defines a model to pre-download and cache. You specify the source URI, model size, and target node groups. Any **InferenceService** in any namespace can use a model cached by a **LocalModelCache** resource if the **storageUri** matches.

LocalModelNamespaceCache

A namespace-scoped variant of **LocalModelCache**. Only **InferenceService** workloads in the same namespace can use the cached model. Use this resource when you need namespace-level isolation for cached models.

LocalModelNodeGroup

A cluster-scoped resource that defines a group of nodes for caching, including storage capacity limits and **PersistentVolume** specifications. Node groups determine which nodes receive cached models and how much storage each node allocates for caching.

LocalModelNode

A cluster-scoped resource that tracks the download status of models on a specific node. The controller creates and updates **LocalModelNode** resources automatically. You can inspect these resources to check the per-node download status of each model.

4.1.3. Cluster-scoped and namespace-scoped caching

Model Cache supports two scoping levels for cached models:

Cluster-scoped (**LocalModelCache**)

Any **InferenceService** in any namespace can use the cached model. Use this scope when multiple teams share the same models and you want to avoid duplicate downloads.

Namespace-scoped (**LocalModelNamespaceCache**)

Only **InferenceService** workloads in the same namespace can use the cached model. Use this scope when you need namespace-level isolation, for example, to restrict model access to specific projects.

Both scoping levels use the same underlying download and storage mechanisms. The choice depends on your organization's access control and multi-tenancy requirements.

4.2. ENABLE MODEL CACHING

You can enable model caching on your cluster so that **InferenceService** pods use locally cached model copies instead of downloading models from remote storage on every startup. Model caching is disabled by default and must be enabled by a cluster administrator through the **DataScienceCluster** custom resource (CR).

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have installed Red Hat OpenShift AI.
- You have enabled the **KServe** model serving platform on your cluster.
- Your cluster has nodes with local Non-Volatile Memory Express (NVMe) storage or **hostPath** volumes available for model caching.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Log in to the OpenShift CLI (**oc**) as a cluster administrator:

```
$ oc login <openshift_cluster_url> -u <username> -p <password>
```

2. Edit the **DataScienceCluster** CR to enable model caching:

```
$ oc edit datasciencecluster <datasciencecluster_name>
```

Replace **<datasciencecluster_name>** with the name of your **DataScienceCluster** resource, for example, **default-dsc**.

3. In the **spec.components.kserve** section, add the **modelCache** configuration:

```
spec:
  components:
    kserve:
      managementState: Managed
      modelCache:
        managementState: Managed
        cacheSize: <storage_capacity>
        nodeNames:
          - <node_name_1>
          - <node_name_2>
```

In the preceding YAML, set the following values.

<storage_capacity>

Specifies the amount of storage to allocate for model caching on each node, for example, **100Gi**. Ensure the cache size accommodates the models you plan to cache.

<node_name_1>, <node_name_2>

Specifies the names of the nodes where models are cached. These nodes must have local storage available.

Alternatively, you can use a **nodeSelector** label selector instead of **nodeNames** to select target nodes dynamically:

```
spec:
  components:
    kserve:
      managementState: Managed
      modelCache:
        managementState: Managed
        cacheSize: <storage_capacity>
        nodeSelector:
          <label_key>: <label_value>
```



NOTE

You cannot specify both **nodeNames** and **nodeSelector** in the same configuration. Use **nodeNames** to select specific nodes by name, or use **nodeSelector** to select nodes by label.

4. Save and close the editor.

Verification

1. Verify that the Red Hat OpenShift AI Operator has labeled the target nodes:

```
$ oc get nodes -l kserve/localmodel=worker
```

The target nodes appear in the output with the **kserve/localmodel=worker** label.

2. Verify that PersistentVolumes are created for the target nodes:

—

```
$ oc get pv -l kserve/localmodel
```

The output shows **PersistentVolumes** for each target node with a **Bound** status.

3. Verify that the **localmodelnode-agent** DaemonSet is running on the target nodes:

```
$ oc get daemonset localmodelnode-agent -n redhat-ods-applications
```

The **DESIRED**, **CURRENT**, and **READY** columns show the same value, matching the number of target nodes.

Next steps

- [Configure node groups for model caching](#)
- [Configure model cache download credentials](#)

4.3. CONFIGURE NODE GROUPS FOR MODEL CACHING

When you enable model caching, the Red Hat OpenShift AI Operator automatically creates a **LocalModelNodeGroup** for all nodes specified in the **DataScienceCluster** CR. You do not need to create a **LocalModelNodeGroup** manually unless you want to cache models on only a subset of those nodes.

For example, if the **DataScienceCluster** CR specifies **node1**, **node2**, and **node3**, the Operator creates a **LocalModelNodeGroup** that includes all three nodes. If you want to cache a model on only **node1** and **node2**, you can create an additional **LocalModelNodeGroup** that includes only those nodes.

A node group specifies the **PersistentVolume** and **PersistentVolumeClaim** templates used to provision storage on each node in the group.

Prerequisites

- You have cluster administrator privileges for your OpenShift cluster.
- You have enabled model caching in the **DataScienceCluster** CR.
- Your target nodes have local NVMe storage or **hostPath** volumes available.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. Optional: To cache models on a subset of nodes, create a YAML file named **node-group.yaml** with the following content:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LocalModelNodeGroup
metadata:
  name: <node_group_name>
spec:
  storageLimit: <storage_limit>
  persistentVolumeSpec:
    accessModes:
```

```

- ReadWriteOnce
capacity:
  storage: <storage_capacity>
local:
  path: /var/lib/kserve/models
nodeAffinity:
  required:
    nodeSelectorTerms:
      - matchExpressions:
          - key: kserve/localmodel
            operator: In
            values:
              - worker
    persistentVolumeReclaimPolicy: Retain
    storageClassName: ""
    volumeMode: Filesystem
  persistentVolumeClaimSpec:
    accessModes:
      - ReadWriteOnce
    resources:
      requests:
        storage: <storage_capacity>
        storageClassName: ""
        volumeMode: Filesystem

```

In the preceding YAML, set the following values.

<node_group_name>

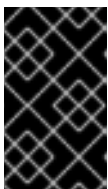
Specifies a descriptive name for the node group, for example, **gpu-nodes**.

<storage_limit>

Specifies the maximum total storage for cached models on each node in this group, for example, **500Gi**. Set this value to accommodate the total size of all models you plan to cache on each node.

<storage_capacity>

Specifies the storage capacity for the PersistentVolume and PersistentVolumeClaim on each node, for example, **500Gi**. This value must be equal to or greater than the **storageLimit**.



IMPORTANT

The **local.path** in the PersistentVolume specification must be **/var/lib/kserve/models**. This path must match the hostPath used by the **localmodelnode-agent** DaemonSet.

2. Create the **LocalModelNodeGroup** resource:

```
$ oc apply -f node-group.yaml
```

Verification

1. Verify that **PersistentVolumes** are created and bound:

```
$ oc get pv -l kserve/localmodel
```

The **PersistentVolumes** show a status of **Bound**.

2. Verify that the **localmodelnode-agent** DaemonSet pods are running on the target nodes:

```
$ oc get pods -n redhat-ods-applications -l app=localmodelnode-agent
```

Each target node has a running agent pod.

3. Verify the node group status:

```
$ oc get localmodelnodegroup <node_group_name> -oyaml
```

The **status.available** and **status.used** fields show the storage capacity.



NOTE

The **status.available** and **status.used** fields might not be populated until models begin caching on the node group. A newly created node group without cached models might show empty status fields.

4.4. CONFIGURE MODEL CACHE DOWNLOAD CREDENTIALS

You can configure credentials that model cache download jobs use to access model artifacts from Hugging Face Hub or S3-compatible object storage. Model Cache uses the **KServe** credential injection mechanism to provide credentials to download jobs. You create a **storage-config** secret in the **kserve-localmodel-jobs** namespace, then reference the secret key in the cache resource's **spec.storage.key** field, or you configure a service account and reference it in the cache resource's **spec.serviceAccountName** field.

Prerequisites

- You have cluster administrator privileges or permissions to create secrets in the **kserve-localmodel-jobs** namespace.
- You have enabled model caching in the **DataScienceCluster** CR.
- You have a Hugging Face Hub access token or S3-compatible storage credentials.
- You have installed the OpenShift CLI (**oc**).

Procedure

- To configure credentials for Hugging Face Hub, create a **storage-config** secret with a JSON-encoded entry for your Hugging Face token:

```
$ oc create secret generic storage-config \
  -n kserve-localmodel-jobs \
  --from-literal=<storage_key>="{\"type\":\"HuggingFace\",\"token\":\"<your_huggingface_token>\"}"
```

In the preceding command, set the following values.

<storage_key>

Specifies a key name for this credential entry, for example, **hf-secret**. You reference this key in the **spec.storage.key** field of the **LocalModelCache** or **LocalModelNamespaceCache** resource.

<your_huggingface_token>

Specifies your Hugging Face Hub access token.

- To configure credentials for S3-compatible object storage, create a **storage-config** secret with a JSON-encoded entry for your S3 credentials:

```
$ oc create secret generic storage-config \
  -n kserve-localmodel-jobs \
  --from-
literal=<storage_key>={"type":"s3","access_key_id":"<access_key>","secret_access_key":"<secret_key>","endpoint_url":"<endpoint_url>","region":"<region>"}
```

In the preceding command, set the following values.

<storage_key>

Specifies a key name for this credential entry, for example, **s3-secret**. You reference this key in the **spec.storage.key** field of the **LocalModelCache** or **LocalModelNamespaceCache** resource.

<access_key>

Specifies the access key ID for your S3-compatible storage.

<secret_key>

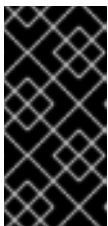
Specifies the secret access key for your S3-compatible storage.

<endpoint_url>

Specifies the endpoint URL for your S3-compatible storage, for example, **https://s3.amazonaws.com**.

<region>

Specifies the region for your S3-compatible storage, for example, **us-east-1**.

**IMPORTANT**

The download job only uses credentials when the **LocalModelCache** or **LocalModelNamespaceCache** resource specifies a **spec.storage.key** or **spec.serviceAccountName** field. Without one of these fields in the cache resource, the download job runs without credentials.

Verification

- Verify that the **storage-config** secret exists in the **kserve-localmodel-jobs** namespace:

```
$ oc get secret storage-config -n kserve-localmodel-jobs
```

The output shows the **storage-config** secret.

4.5. MODEL CACHE CONFIGURATION PARAMETERS

You can configure model caching by setting the **ModelCacheSpec** fields in the **DataScienceCluster** custom resource (CR). These parameters control how model caching is enabled, which nodes are used for caching, and how much storage is allocated.

4.5.1. DataScienceCluster model cache parameters

The following table describes the **spec.components.kserve.modelCache** fields in the **DataScienceCluster** CR.

Table 4.1. **ModelCacheSpec** fields

Field	Type	Required	Description
managementState	string	Yes	Controls whether model caching is enabled. Set to Managed to enable model caching or Removed to disable it.
cacheSize	Quantity	Yes	The amount of storage to allocate for model caching on each target node, for example, 100Gi . Ensure this value accommodates the total size of models you plan to cache.
nodeNames	[]string	No	A list of specific node names where models are cached. Cannot be used together with nodeSelector .
nodeSelector	map[string]string	No	A label selector that dynamically selects nodes for model caching. Cannot be used together with nodeNames .



NOTE

A Common Expression Language (CEL) validation rule enforces mutual exclusivity between **nodeNames** and **nodeSelector**. You must specify exactly one of these fields. Although each field is individually optional, at least one must be present to identify the target nodes for model caching. You cannot specify both fields in the same configuration.

Example DataScienceCluster configuration

```
apiVersion: datasciencecluster.opendatahub.io/v1
kind: DataScienceCluster
metadata:
  name: default-dsc
spec:
  components:
    kserve:
      managementState: Managed
      modelCache:
        managementState: Managed
        cacheSize: "200Gi"
        nodeNames:
```

- gpu-node-1
- gpu-node-2

4.5.2. Model size reference for capacity planning

The following table lists approximate storage requirements for common LLM models. Use these values to plan the **cacheSize** and node group **storageLimit** settings.

Table 4.2. Approximate model sizes

Model	Precision	Approximate size	Notes
Llama 3 8B	FP16	16 Gi	Suitable for smaller GPU nodes
Llama 3 70B	FP16	140 Gi	Requires nodes with large local storage
Mixtral 8x7B	FP16	94 Gi	Mixture-of-experts model
Mixtral 8x22B	FP16	282 Gi	Large mixture-of-experts model
Granite 3.2 8B	FP16	16 Gi	IBM Granite model



NOTE

Model sizes vary significantly depending on the quantization method and precision format. The sizes listed are approximate values for FP16 (half-precision) model weights. Quantized variants such as INT8 or INT4 are substantially smaller. Verify the actual size of your specific model variant before setting cache parameters.

4.6. TROUBLESHOOT MODEL CACHING ERRORS

You can diagnose and resolve common issues that occur when configuring and using Model Cache. The following sections describe symptoms, causes, and solutions for known issues.

Download job fails with **Init:OOMKilled**

The download job's init container runs out of memory. This typically occurs when the **modelSize** value in the **LocalModelCache** resource is too small for the actual model.

To resolve the issue, delete the **LocalModelCache** resource and create a new one with a larger **modelSize** value that matches the actual size of the model.

Download jobs appear duplicated

Multiple download jobs might be created for the same model and node. This is expected behavior and does not indicate a problem. The **localmodel-controller-manager** creates new download jobs when reconciling cache state. Duplicate jobs do not cause data corruption.

Permission denied errors on download jobs

Download jobs or the **localmodelnode-agent** fail with permission denied errors when accessing the local storage path. This typically occurs when the node's file system permissions do not allow the download job to write to **/var/lib/kserve/models**.

The Model Cache feature requires the **kserve-localmodel-permissions-scc** SecurityContextConstraints (SCC) to be present on the cluster. This SCC grants the necessary Linux capabilities: **CHOWN**, **DAC_OVERRIDE**, and **FOWNER**. You can verify that the SCC exists by running the following command.

```
$ oc get scc kserve-localmodel-permissions-scc
```

If the SCC is missing, verify that model caching is enabled in the **DataScienceCluster** CR and that the Red Hat OpenShift AI Operator has reconciled successfully.

localmodelnode-agent DaemonSet not scheduling

The **localmodelnode-agent** DaemonSet pods are not scheduled on target nodes. This can occur when nodes do not have the required **kserve/localmodel=worker** label, or when node taints prevent scheduling.

Verify that target nodes have the correct label:

```
$ oc get nodes -l kserve/localmodel=worker
```

If nodes are missing the label, verify that the **nodeName**s or **nodeSelector** values in the **DataScienceCluster** CR match the target nodes.

PersistentVolumeClaim binding failures

PersistentVolumeClaims for model caching remain in **Pending** state. This can occur when **PersistentVolumes** are not available or when the storage class configuration is incorrect.

Verify that **PersistentVolumes** exist and are available:

```
$ oc get pv -l kserve/localmodel
```

If **PersistentVolumes** are missing, verify that model caching is enabled in the **DataScienceCluster** CR. If you are using a custom **LocalModelNodeGroup**, verify that the **persistentVolumeSpec** and **persistentVolumeClaimSpec** configurations are correct.

Download stuck in NodeDownloadPending state

The download job is created but never starts downloading. This can indicate credential issues or network connectivity problems.

Check the download job status:

```
$ oc get jobs -n kserve-localmodel-jobs
```

View the job logs for error messages:

```
$ oc logs job/<job_name> -n kserve-localmodel-jobs
```

Verify that download credentials are configured correctly in the **kserve-localmodel-jobs** namespace:

```
$ oc get secrets -n kserve-localmodel-jobs
```

Namespace PSA elevation to privileged

The **kserve-localmodel-jobs** namespace is configured with a **privileged** Pod Security Admission (PSA) level. This is expected behavior. The download jobs and permission-fix init containers require elevated privileges to set file ownership and permissions on the local storage path.

InferenceService does not use cached model

The **InferenceService** pod downloads the model from remote storage instead of using the local cache. This occurs when the **storageUri** in the **InferenceService** does not match the **sourceModelUri** of the **LocalModelCache** or **LocalModelNamespaceCache** resource.

Verify that the **storageUri** exactly matches the **sourceModelUri**:

```
$ oc get localmodelcache <cache_name> -o jsonpath={.spec.sourceModelUri}
```

Compare the output with the **storageUri** in your **InferenceService**. The values must match, or the **InferenceService storageUri** must be a subdirectory of the **sourceModelUri**.

Also verify that the model download is complete and the node running the **InferenceService** pod has a **NodeDownloaded** status.